10 Event-driven applications and functions

This chapter covers

- Understanding event-driven architectures
- Using RabbitMQ as a message broker
- Implementing functions with Spring Cloud Function
- Processing events with Spring Cloud Stream
- Producing and consuming events with Spring Cloud Stream

In the previous chapters, we worked on a system of distributed applications that interact according to the request/response pattern, a type of synchronous communication. You saw how to design the interaction both in an imperative and a reactive way. In the first case, processing threads would block, waiting for a response from an I/O operation. In the second case, threads would not wait. A response would be processed by any available thread asynchronously once it was received.

Even if the reactive programming paradigm lets you subscribe to producers and process the incoming data asynchronously, the interaction between the two applications is synchronous. The first application (the client) sends a request to the second one (the server) and expects a response to arrive in a short time. How the client processes the response (imperative or reactive) is an implementation detail that doesn’t affect the interaction itself. No matter what, a response is expected to arrive.

Cloud native applications should be loosely coupled. The microservices expert Sam Newman identifies a few different types of coupling, including